

Capacity Estimates and Data Models That Matter

Back-of-the-envelope estimates are decision tools, not predictions of exact production behavior. Translate user activity into requests, bytes, concurrency, and storage, then connect the dominant costs to data modeling. Keep average, peak, logical data, replicas, and overhead separate.

For: Candidates who can draw service diagrams but need to connect scale assumptions, units, retention, indexes, and access patterns to concrete architectural decisions.

Suggested pace: 2 weeks of numerical and schema exercises | Intermediate

What you will be able to do

- Convert daily activity into request rate without confusing units.
- Estimate storage, bandwidth, concurrency, and retention costs explicitly.
- Choose keys and indexes from reads, writes, and ownership boundaries.
- Identify sensitivity to peak factors, payload size, fan-out, and hot partitions.

Build the right mental model

Name every unit

Requests/day divided by 86,400 gives average requests/second. Bits and bytes differ by eight; GB and GiB use different bases. A capacity estimate is only useful when units, assumptions, and the relevant time window are visible.

Peak and fan-out change the load

A user request can trigger multiple database reads or messages. A global average hides regional peaks and hot entities. Model amplification and skew separately rather than multiplying every number by an unexplained safety factor.

Access patterns shape data

Identify the entity key, query predicates, ordering, and transaction boundary. A storage choice should support those operations and retention needs. Denormalized views can improve reads but require a defined update and reconciliation mechanism.

Study with evidence.

Use a small worksheet with assumptions, formula, units, and design consequence for each estimate. Vary one assumption to see whether the architecture changes. Do not add more decimal places than the uncertainty supports.

Days 1-3: Rates and payloads

Estimate daily active users, actions per user, read/write ratio, and average payload. Convert to average rate and introduce a justified illustrative peak factor. Separate client requests from internal work.

Deliverable and check: A request-rate table with consistent units and a note explaining the largest uncertainty.

Days 4-6: Storage and retention

Estimate logical record size, daily growth, retention, replicas, indexes, and compression assumptions. Distinguish hot queryable data from archive and account for deletion or compaction lag.

Deliverable and check: A storage budget that does not confuse source bytes with fully replicated indexed capacity.

Days 7-10: Keys and queries

Write important queries before choosing partition keys or indexes. Check whether a key causes hotspots, whether pagination has stable ordering, and which updates must be atomic together.

Deliverable and check: A schema and access-pattern table with one hot-key scenario and one transaction invariant.

Days 11-14: Sensitivity and validation

Change peak traffic, payload size, and retention independently. Estimate concurrency using arrival rate and time in system under suitable steady-state assumptions. Identify what a load test must measure before deployment.

Deliverable and check: A concise design note showing which assumptions matter and what evidence would disconfirm the estimate.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Size a service from daily volume

Assume 20 million requests/day, a 10x peak factor, 200 ms mean time in system at peak, and 1 KB of durable event data per request. These are hypothetical design inputs, not claims about a real product.

A worked approach

1. Compute average load: $20,000,000 / 86,400$ is about 231 requests/second. A 10x illustrative peak is about 2,315 requests/second. Do not confuse daily request count with instantaneous rate.
2. Using $L = \lambda \times W$ under the relevant steady-state assumptions, about $2,315 \times 0.2$ gives 463 requests in the system on average at that load. This is not a direct thread count or connection-pool prescription.
3. At decimal 1 KB per event, raw growth is about 20 GB/day. Thirty days is about 600 GB before replication, indexes, metadata, and compression. Three full copies alone raise that raw component to about 1.8 TB.

Expected result

The calculation supplies approximate rate, concurrency, and storage dimensions with clear assumptions. It points to measurements needed for queueing, connection limits, replication overhead, and actual payload distribution.

Verify the result

- Are decimal and binary storage units labeled consistently?
- Are replicas and indexes added rather than silently assumed free?
- Does the peak factor represent a plausible workload shape rather than certainty?

Extend the exercise

Suppose 1% of requests target one customer and each request fans out to ten reads. Recalculate the hot partition's work and explain why evenly dividing total traffic across shards would be misleading.

Make the learning stick.

Can I choose a database from data volume alone?

No. Query patterns, transaction needs, consistency, latency, retention, operational expertise, and failure behavior also matter. Large data does not automatically imply a particular database family.

Does Little's Law determine the worker count?

It relates average arrival rate, time in system, and average work in the system under its assumptions. Worker sizing also depends on CPU, blocking, service-time distribution, queueing, and resource limits.

How precise should estimates be?

Use enough precision to guide a decision and show unit correctness. Most inputs are uncertain, so ranges and sensitivity analysis are often more honest than a long exact-looking number.

Avoid these traps

- Using average traffic as if bursts and hotspots do not exist.
- Ignoring index, replication, and retention overhead.
- Choosing partition keys before listing important queries.

Ready for the next step?

- Each estimate names units and assumptions.
- Read/write amplification and skew are considered.
- Data keys support actual query and ownership needs.
- A validation workload can test the controlling assumptions.

Reliable next reads

- [Azure data partitioning guidance](#)
- [PostgreSQL indexes](#)
- [Google SRE workbook](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.